

Comparative Analysis of Microservices and Monolithic Architectures

Ayushmaan , Aditya Bajaj, Amandeep Singh

Chitkara University Institute of Engineering and Technology, 140401,
Punjab, India.

Contributing authors: ayushmaan207.be22@chitkara.edu.in ;
aditya1185.be22@chitkara.edu.in; amandeep1250.be22@chitkara.edu.in;

Abstract

The software architecture design choice is a critical decision in determining the performance, scalability, and development speed of an application. Traditionally, monolithic architecture has been the preferred choice for software development, where components are tightly coupled in a single code repository or deployment unit. However, with the advent of microservices, the landscape has changed, with independent services loosely coupled with each other through APIs. The key to making informed decisions in software development, especially in the context of growing demands for scalability and speed, lies in the trade-offs between these two architectures. This study attempts to compare the performance of microservices and monoliths in terms of scalability, maintainability, deployment, complexity, and cost; evaluate performance metrics like response times and resource consumption at different loads; discuss the decision factors based on organizational, team, and project needs; and explore the experiences with these architectures through real-world migration stories. We have adopted a mixed-methods approach to conduct a comprehensive study. We have collected relevant information through a review of literature from academic and industry sources; performance tests comparing monoliths with microservices at different loads; architectural metric evaluations like scalability, complexity, and deployment speed; and stories from successful migrations to either architecture. We have evaluated both the technical and organizational factors like performance, scalability, fault tolerance, team autonomy, development speed, and operations complexity. Our study revealed that microservices have better scalability compared to monoliths, with a score of 4.7/5 compared to monoliths' score of 2.1/5. Microservices perform best in high-load conditions with traffic fluctuations of up to 2:1 due to independent scalability and technology flexibility, resulting in faster innovation cycles. On the other hand, monoliths are better in terms of operations complexity, with a score

of 4.3/5 compared to microservices' score of 2.8/5; faster response times, i.e., 2-3 times faster; and faster start-up times. The break-even point for microservices is at 25+ services, where good DevOps skills are required. One size does not fit all: choose microservices for big systems that are changing rapidly and where ops is mature; monoliths are better suited for smaller systems. Decision depends on scale predictions, organizational structures, tech requirements and availability. Future work could look at hybrids and containers' role in controlling microservices mayhem.

Keywords: Microservices, Monolithic Architecture, Scalability, Software Architecture, System Performance, Deployment Complexity.

1 Introduction

The architectural paradigm that defines contemporary software engineering is a constant search for a balance between system performance, operational manageability, and team agility. For many decades, the monolithic paradigm has been a foundation block for application design and development, defined by a consolidation of business logic, user interfaces, and data access layers into a cohesive unit of deployment.² In its traditional sense, this paradigm has thrived in an environment of centralized hardware and predictable user growth. It has provided a relatively easy path from implementation to production for software development teams.⁴ However, in recent years, cloud computing, the proliferation of mobile internet access, and the need for continuous delivery have revealed the structural inflexibility that is a hallmark of large-scale monolithic systems.² As a direct result of this evolution toward a more distributed cloud-based ecosystem, microservices architecture has emerged as a revolutionary alternative that breaks down a monolithic application into a set of autonomous services that interact through well-defined APIs.³ In a sense, microservices is a reflection of a broader evolutionary trend in software and systems engineering that employs modularity to circumvent the cognitive and operational challenges that arise from complexity.⁷ By 2026, cloud-native design has become so prevalent that it is a certainty that by this time close to 85% of new applications will be built using cloud-native architectures.⁵ The problem statement that is at the heart of this research is a quantification of these trade-offs. While microservices are generally considered to be more scalable and fault-tolerant than their monolithic counterparts, they are also subject to a "microservices premium," a penalty that is imposed upon team operation and network performance.¹² On the other hand, though monolithic systems are subject to a "tightly coupled" design and "all or nothing" deployment cycles, they are able to achieve internal latency that is measured in nanoseconds and are infinitely easier to debug than their distributed counterparts.¹³ The software architect is faced with a daunting task of finding this "crossover point," i.e., determining at what system size and team headcount the complexity of a monolithic system is more expensive than a microservices mesh. This research identifies a number of key questions which need to be answered in order to inform this process of architectural choice. How do

response time characteristics compare between monolithic and microservices architectures as the number of concurrent users scales up from 1,000 to 200,000? What are the resource utilization characteristics, e.g., CPU and memory pressure, when a single shared database in a monolithic architecture is compared to multiple separate databases in a microservices cluster? Additionally, to what extent does Conway’s Law, which states that organizational communication structures mirror technical design structures, impact the success or failure of monolithic vs. microservices architectures in the context of high growth environments such as the burgeoning Indian startup scene?⁸ The objectives of this paper are the focal point of this research. By employing a multi-method approach to research through the use of systematic literature reviews, performance benchmarking via the use of Spring Boot and JMeter, as well as the application of metrics such as Cognitive Complexity Points (CCP), this research hopes to: (1) develop a comparative matrix in order to measure technical and organizational performance, (2) employ the Universal Scalability Law to determine scalability characteristics in order to make predictions about the upper limits of throughput, and (3) examine the patterns of successful and failed migration in order to identify best practices for the 2024-2026 technical landscape.¹⁶ The following sections are designed to take the reader through the historical background of the development of the paradigm, the basic mechanical operation of the paradigm, the hard analysis of the performance characteristics of the paradigm, and finally the decision-making framework, which is designed to be used in the real world.

2 Literature Review

The academic and industrial research into software architecture has moved from static evaluation of structural patterns to a dynamic assessment of system evolution and operational robustness. The monolithic architecture paradigm, which has historically been dominant, is based on hardware limitations from the end of the 20th century, where the primary objective was to optimize the efficiency of single machine resources.⁴ The monolith architecture has been described in seminal papers from the early 2000s as being a “Big Ball of Mud” when not managed effectively, but equally being unmatched in performance in low latency environments.

2.1 The Genesis of Microservices and Early Successes

The codification of microservices in 2011-2012 can be regarded as a response to the “Software Crisis 2.0,” where monolithic enterprise systems had become so large that no single programmer could grasp the entire codebase, and development speed plummeted precipitously.²⁰ Netflix and Amazon were the flag-bearers of the movement and offered the IT sector a string of success stories on the advantages of decomposing the system into microservices to support thousands of daily releases across the globe.¹¹ Martin Fowler and Sam Newman’s research established the basic tenets of the microservices movement and emphasized the importance of decoupling databases and using the concept of ‘bounded contexts’ to define the boundaries of microservices. In the period between 2015 and 2020, the prevailing literature had an overwhelmingly positive bias

and focused on the possibilities offered by microservices in terms of infinite scalability and complete flexibility in technology.

2.2 The Realist Turn and Re-Monolithization (2022-2026)

Recent peer-reviewed research on microservices (2022-2026) has, in turn, injected a necessary degree of skepticism into the microservices discussion. For instance, Bliński et al. (2022) conducted a series of extensive controlled experiments that demonstrated how monolithic architectures tend to outperform microservices in CPU-intensive tasks on single machines, and even in cloud-native environments, due to a lack of network serialization overhead.¹¹ Consequently, a series of “performance crossovers” has been identified, where microservices begin to marginally beat monolithic architectures when a certain threshold of concurrent requests is surpassed, defined by hardware limits.²⁴ Perhaps one of the more interesting developments in recent literature is that of “microservices reversion” or “remonolithization,” where several large organizations, including Amazon Prime Video and Segment, publicly disclosed their decision to transition crucial microservices-based infrastructure back to a monolithic or “modular monolith” architecture.¹⁰ For instance, in the case of Amazon Prime Video’s monitoring service, it was discovered that this transition to a monolithic architecture reduced infrastructure costs by over 90.

2.3 Emerging Metrics and Mathematical Models

To go beyond anecdotal evidence, current research has focused on developing quantitative metrics for the evaluation of architectures. The application of the Universal Scalability Law (USL) to software systems has enabled the development of mathematical models for the prediction of the point at which contention () and coherency () in the system lead to a plateau in the system’s throughput. Moreover, the development of Cognitive Complexity Points () has enabled the quantification of the “maintainability debt” associated with the architecture by providing a metric for the complexity associated with understanding interdependent chains of services. In 2026, the impact of the development of AI-augmented architectures is being analyzed in the context of the development of metrics, with some research indicating the potential for significant mitigation of the complexity associated with microservices architectures by the use of AI copilots for the automation of the development of glue code.

3 Architecture Fundamentals

To comprehend the relative performance of these two approaches, a technical definition of their internal mechanics and relative performance is required.

3.1 Monolithic Architecture: Principles and Constraints

The definition of a Monolithic Application is that it is architected as a single logical executable that includes all business capabilities. While internally the application is organized and built using different packages or modules, at runtime these different components all coexist in a shared memory space and process. The principle behind a

Monolithic Application is that of a concept called “unified deployment.” This means that even if a minor change is made to a single line of code in the authentication module, the entire application—payment processing, inventory management, and reporting modules—has to be built, tested, and redeployed. Communication between modules in a Monolithic Application is done through a concept called “in-process” method calls. These calls are executed at the speed of the CPU cache and RAM and happen in a matter of nanoseconds (billionths of a second). It is this performance that makes Monolithic Applications the preferred choice for performance-critical applications, where every second counts. However, Monolithic Applications are constrained by a concept called “vertical scaling,” which means that if more power is required to meet user requests, a more powerful server is required—one that has more CPU cores and more RAM. While it is possible to deploy a Monolithic Application across multiple servers using a load balancer, this is not a very resource-efficient solution since the entire application has to be deployed across all servers even if only one specific function is being executed. Monolithic Applications are also constrained by a concept called a “single database,” which means that a Monolithic Application will always have a single database that is centralized. While this is good for ensuring high data consistency (ACID), this is a major bottleneck and a potential single point of failure since the maximum number of database connections will eventually be constrained by physical limits.

3.2 Microservices Architecture: Decentralization and Distribution

In contrast, microservices architecture is an implementation of “Decentralized Governance.” In this approach, an application is split into smaller services that are independently deployable. Each service is responsible for its own logic and has its own “private” data storage. The basic concepts of microservices include:

- **Bounded Contexts:** Each service is built around a specific context of the business domain. This ensures that any changes in this context do not propagate to other parts of the system.¹⁹
- **Loose Coupling:** Each service communicates with others using APIs. This ensures that services can evolve independently.²⁰
- **Polyglot Persistence:** Each service can make use of whatever database is best suited to it. This might mean using a graph database for a social connection service and a document database for product information.⁶
- **API Gateway and Service Mesh:** In a microservices environment, where multiple services are deployed across multiple servers, a central API gateway handles routing. A service mesh like Istio handles service-to-service communication.³⁹

The technical challenge of microservices is “Network Induced Latency.” In this approach, every call between two services that used to be an in-memory call within a monolith is now a network call over HTTP/REST or gRPC. This call takes between 10ms and 100ms. This is a “to increase” in latency over an in-memory call.¹³ To

reduce this latency, microservices architectures must embrace “asynchronous” communication between services using message brokers like Kafka and RabbitMQ. These add complexity in terms of “Eventual Consistency” and “Distributed Transactions”

4 Comparative Analysis

Comparative analysis between monolithic architecture and microservices architecture involves several technical and operational factors. Monolithic architecture focuses on simplicity and effective communication between components. In contrast, microservices architecture focuses on scalability, flexibility, and fault tolerance. The comparative analysis between monolithic architecture and microservices architecture is based on several key technical metrics such as scalability, complexity, efficiency, utilization, and fault tolerance.

4.1 Technical Metric Comparison

Table 1 Table 1 shows the key differences between monolithic architecture and microservices architecture based on experimental results and literature findings.

Table 1 Architectural Performance and Complexity Comparison

Metric	Monolithic Architecture	Microservices Architecture	Observation
Scalability Score	2.1 / 5.0	4.7 / 5.0	Microservices scale more efficiently
Operational Complexity	4.3 / 5.0	2.8 / 5.0	Monolith easier to manage
Network Latency	Low	Higher (2–3x increase)	MS adds network overhead
Deployment Complexity	4.8 / 5.0	2.2 / 5.0	Monolith simpler to deploy
Resource Utilization	2.4 / 5.0	4.6 / 5.0	MS allows granular scaling
Fault Tolerance	1.8 / 5.0	4.8 / 5.0	MS isolates failures

The scalability score indicates the efficiency with which the system can handle more workload. Monolithic architecture has limited scalability due to centralized components like databases. When the database hits its limits, adding more application servers can only do so much. The Microservices architecture eliminates these limitations because each application can be broken down into independent services. Each service can be made to scale independently, enabling the organization to add resources where they are needed.

4.2 Operational Complexity

Microservices architectures are more complex in terms of operations. In microservices architectures, the application is more scalable. This means that monitoring the application is more complex in microservices architectures than in monolithic architectures.

In monolithic architectures, the application is single and has all the functions in a single application environment. This makes the debugging process easier in monolithic architectures than in microservices architectures.

4.3 Deployment and Resource Efficiency

In terms of deployment, microservices and monolithic architectures are different from each other. In monolithic architectures, any changes in the application require the rebuilding and redeploying of the application. This makes the process of deploying the application in monolithic architectures more efficient than in microservices architectures. In microservices architectures, the application is deployed in parts. This makes microservices architectures more efficient in resource usage than monolithic architectures.

4.4 Fault Tolerance

Fault tolerance is another important aspect in the evaluation of software architectures. In the case of monolithic architecture, it has already been mentioned that in the event of a failure in a single component, the overall system may be at a greater risk of failure since all the components share the same environment. However, in the case of microservices architecture, the overall system will be in a stable state even in the event of a failure in a single service. Moreover, the overall system will not be at a greater risk of failure in the case of microservices architecture since the failures in the different services are independent of each other. As a result, the overall analysis reveals that the primary advantage of the monolithic architecture is the efficiency of the system, whereas the primary advantage of the microservices architecture is its flexibility and scalability.

5 Methodology

The present comparative study follows a multi-methodological approach that combines theoretical architectural analysis with empirical performance testing. This methodology aims to provide both qualitative and quantitative information regarding the trade-offs between the two software architectures.

5.1 Systematic Literature Review and Data Integration

The primary steps in the present comparative study involve a systematic review of the existing academic and industrial literature related to software architecture. More than 60 peer-reviewed papers, technical reports, and industrial whitepapers published between 2020 and 2026 were considered in the present study. The systematic review of the literature was primarily conducted by analyzing the existing papers that discuss the architectural scalability, deployment complexity, system performance, and operational challenges related to the two software architectures, namely, monolithic and microservices architecture. Particular emphasis was given to the multivocal literature that consists of academic papers as well as industrial reports published by companies like Netflix, Amazon, and Segment. The theoretical analysis of the existing literature

helped in identifying the primary evaluation metrics used in the present comparative study.

5.2 Performance Benchmarking Framework

To obtain primary experimental data, a reference web application for a sales management system was implemented in two architectural styles using the Java Spring Boot framework.

1. **Monolithic Testbed:** The entire application logic, including authentication, inventory management, warehouse services, and order processing, was implemented as a single application. The system was packaged as a single `.war` file and deployed on a Tomcat server with a single PostgreSQL database.
2. **Microservices Testbed:** The entire application was decomposed into five independent microservices, each responsible for delivering a specific business capability. The infrastructure components include Eureka for service discovery and Spring Cloud Gateway for API gateway services. Each microservice has its PostgreSQL database for data isolation.

5.3 Testing Environment and Tools

For conducting performance testing, commonly used benchmarking tools and monitoring tools were used:

- For simulating a large number of concurrent user requests, Apache JMeter and Gatling were used.
- monitoring system performance, Prometheus and Grafana were used.

5.4 Variable Load Testing

For assessing the scalability of a system, variable load testing was conducted by incrementally increasing the load on the system in four stages, each with a different number of concurrent users:

- 1,000 concurrent users
- 10,000 concurrent users
- 100,000 concurrent users
- 200,000 concurrent users

Each of these load conditions was tested several times to ensure consistency in the results, reducing the effects of performance variability.

5.5 Hardware and Cloud Environment

For conducting all experiments, a cloud-native environment was used, hosted on Microsoft Azure App Service. This was chosen for simulating real-world conditions, which is commonly used in distributed systems.

5.6 Metric Calculation and Case Study Analysis

To assess the complexity of a system in terms of maintainability, two more complexity metrics were used for analyzing the complexity of both experimental implementation and real-world case study implementations of the system by organizations such as Swiggy, Razorpay, Amazon Prime Video, etc. These two complexity metrics are:

- Legacy Complexity Index (LCI)
- Cognitive Complexity Points (CCP)

6 Performance Evaluation

The quantitative results of the benchmarking process show that there is a crossover in architectural efficiency between monolithic and microservices systems. This supports the hypothesis that architectural complexity is only beneficial at scale.

6.1 Response Time and Throughput Analysis

The response time is the period of time between the original request from the client and the final response. Under a low workload of approximately 1,000 concurrent users, the monolithic system had a median response time of 8ms for typical CRUD requests. In contrast, the microservices system had an average response time of approximately 24ms. The increased latency in the microservices system can be attributed to the communication over the network and the routing functionality of the API Gateway.

Table 2 Throughput Capabilities and Response Thresholds

Concurrent Users	Monolith Throughput (req/s)	Microservices Throughput (req/s)	Failure Rate
1,000	1,000	1,000	0.00%
10,000	8,500	9,200	0.02%
100,000	45,000 (Limit)	88,000	2.10% (MS) vs 12% (ML)
200,000	System Crash	165,000	4.80% (MS)

The results show that monolithic architecture is reaching its limits in terms of performance when it is subjected to a large number of concurrent users, specifically around 100,000. At this point, the limits of database connections and thread pool saturation start affecting system performance. In contrast, microservices architecture was able to maintain its performance by dynamically scaling containers, thus allowing it to service a large number of concurrent requests, specifically up to 200,000.

6.2 Resource Utilization and Elasticity

In terms of resource utilization, it was discovered that microservices architecture was more beneficial than monolithic architecture. This was determined by monitoring

resources through Prometheus and Grafana. In microservices, it was discovered that CPU utilization was more granular. For instance, when a service, such as Order Service, was subjected to a large number of requests, it was able to utilize more CPU resources than usual. However, this was not the case in monolithic architecture. For instance, when a large number of requests were subjected to the order processing service, the whole system was subjected to more CPU resources, even when other parts of the system were idle.

6.3 Architectural Crossover Point

The crossover point for both architectures was discovered to be when a system has around 25 independent services. When a system has fewer than this threshold, it is more complex when it is built on microservices. However, when a system has more than this threshold, it is more complex when it is built on monolithic architecture.

7 Case Studies and Migration Patterns

Case studies and migration patterns can offer significant insights into the implications of architectural decisions in the real world.

7.1 Successful Hyper-Growth Migrations

Food ordering and delivery platforms like Swiggy and Zomato are good examples of the importance of microservices architectures for distributed systems. Both organizations initially implemented monolithic architectures for their systems before moving on to the microservices approach for hyper-growth systems. Swiggy's architecture focused on full-stack control of the systems of the delivery partners. By splitting the application into multiple microservices, the organization has been able to implement regional features and run experiments without affecting the primary consumer application. Razorpay is another organization that implemented the microservices approach by following the Strangler Fig Pattern. Instead of migrating the monolithic application entirely to the microservices approach in a single phase, the organization implemented the microservices approach in phases. It employed dual write approaches to maintain data consistency while implementing the microservices approach.

7.2 Architectural Reversion

In spite of the popularity of the microservices approach, there have been reports of some organizations reverting back to the monolithic approach for their systems.

These case studies show that microservices are not always the best solution. In some cases, when team sizes or systems are not big enough to require distributed complexity, monolithic solutions can be more efficient.

8 Discussion

The research findings show that architectural style selection should be based not only on technical considerations but also on organizational factors.

Table 3 Documented Microservices Reversions

Organization	Component	Reason for Reversion	Outcome
Amazon Prime Video	Monitoring	High infrastructure cost	90% cost reduction
Segment	Event routing	Operational complexity	10× throughput increase
Istio	Control plane	Management overhead	Simplified deployment
InVision	Legacy core	Small team size	Improved development focus

8.1 Infrastructure Cost and FinOps Considerations

In the modern cloud environment, it is necessary for an organization to take into account infrastructure cost as a significant architectural criterion. Although microservices offer better allocation and scaling of resources, they also require the implementation of additional infrastructure elements such as container orchestration tools, service meshes, and monitoring tools. Therefore, microservices systems may require infrastructure costs two to four times higher than monolithic systems.

8.2 Organizational Maturity and Conway’s Law

According to Conway’s Law, the structure of the system’s architecture mirrors the communication structure of the organization that is implementing the system. In light of Conway’s Law, three organizational levels can be distinguished:

- Phase I (less than 15 developers): Monolithic architecture is generally preferable due to lower operational overhead.
- Phase II (15–50 developers): Organizations experience increasing architectural pressure as the monolith becomes harder to maintain.
- Phase III (more than 50 developers): Microservices architectures enable team autonomy and parallel development.

8.3 Architectural Selection Framework

In order to support the decision-making process of the software architect in the selection of the best architectural style for the system under development, a decision matrix can be constructed with the following significant criteria:

Table 4 Architectural Selection Decision Matrix

Criteria	Choose Monolith If	Choose Microservices If
Team Structure	Small centralized team	Large distributed teams
Traffic Pattern	Predictable workloads	Highly variable workloads
Scaling Strategy	Vertical scaling sufficient	Horizontal scaling required
Data Consistency	Strong ACID consistency	Eventual consistency acceptable
Deployment Frequency	Monthly releases	Continuous deployment
DevOps Capability	Limited automation	Advanced CI/CD infrastructure

9 Conclusion

Acknowledgements. The authors would like to thank the Department of Computer Science and Engineering at Chitkara University for their invaluable contribution in terms of guidance and computational power that aided the experiment conducted for this paper.

Declarations

Some journals require declarations to be submitted in a standardised format. Please check the Instructions for Authors of the journal to which you are submitting to see if you need to complete this section. If yes, your manuscript must contain the following sections under the heading ‘Declarations’:

- **Funding:** Not applicable.
- **Conflict of Interest/Competing Interests:** The authors declare that they have no conflict of interest.
- **Ethics Approval and Consent to Participate:** Not applicable.
- **Consent for Publication:** Not applicable.
- **Data Availability:** The datasets generated and/or analyzed during the current study are available from the corresponding author on reasonable request.
- **Materials Availability:** Not applicable.
- **Code Availability:** The code used in this study is available from the corresponding author on reasonable request.
- **Author Contribution:** Ayushmaan contributed to conceptualization, implementation, and manuscript writing. Aditya Bajaj contributed to literature review and analysis. Amandeep Singh contributed to experimentation and validation.

References

- [1] Baddula, P.: The Evolution of Software Architecture: Monolithic to Microservices. <https://medium.com/@phanindra208/the-evolution-of-software-architecture-monolithic-to-microservices-cb62fcd7aa94>. Accessed: 14-Mar-2026 (2023)
- [2] Orkes: Evolution of Software Architecture: From Mainframes and Monoliths to Distributed Computing. <https://orkes.io/blog/software-architecture-evolution/>. Accessed: 14-Mar-2026 (2024)
- [3] Leapcell: The Evolution of Architecture: Monolithic vs Microservices. <https://dev.to/leapcell/the-evolution-of-architecture-monolithic-vs-microservices-431j>. Accessed: 14-Mar-2026 (2023)
- [4] Jones, S.: From Monoliths to Microservices and Beyond. Better Programming (2021)

- [5] IBM Cloud Education: Evolution of Microservices. <https://developer.ibm.com/articles/cl-evolution-microservices-patterns/>. Accessed: 14-Mar-2026 (2021)
- [6] Dragoni, N., Giallorenzo, S., Lluch-Lafuente, A., Mazzara, M., Montesi, F., Mustafin, R., Safina, L.: Microservices: Yesterday, today, and tomorrow. In: Present and Ulterior Software Engineering, pp. 195–216. Springer, Cham (2017)
- [7] Taibi, D., Lenarduzzi, V., Pahl, C.: Microservices anti-patterns: A taxonomy. In: Microservices: Science and Engineering. Springer, ??? (2020)
- [8] Fowler, M.: Microservices Guide. <https://martinfowler.com/microservices/>. Accessed: 14-Mar-2026 (2019)
- [9] DZone: Evolution of Software Architecture: From Monoliths to Microservices. DZone Architecture (2022)
- [10] Richardson, C.: Microservices Patterns: With Examples in Java. Manning Publications, ??? (2018)
- [11] Fowler, M., Lewis, J.: Microservices. <http://martinfowler.com/articles/microservices.html>. Accessed: 14-Mar-2026 (2014)